

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

christian.holz@lehre.dhbw-stuttgart.de



Recap Initialisierungslisten

```
class Foo
{
public:
    Foo(int x) : m_x(x) {}
private:
    int m_x;
};
```

```
class Foo
{
public:
    Foo(int x)
    {
        m_x = x;
    }
private:
    int m_x;
};
```

- **Frage:** Was sind die Unterschiede der beiden Initialisierungsmethoden?
- **Frage:** Was sind die Vor- und Nachteile?

Recap Aufgabe: Objektorientierung - Einführung

Verwenden Sie die Aufgabenbeschreibung aus dem Git Rep:

2_Objektorientierung/212_Konstrukturen/Rectangle/TASK_RECTANGLE.md

Testen Sie Ihre Klasse durch Aufruf in der `main` Funktion.

Kapitelübersicht

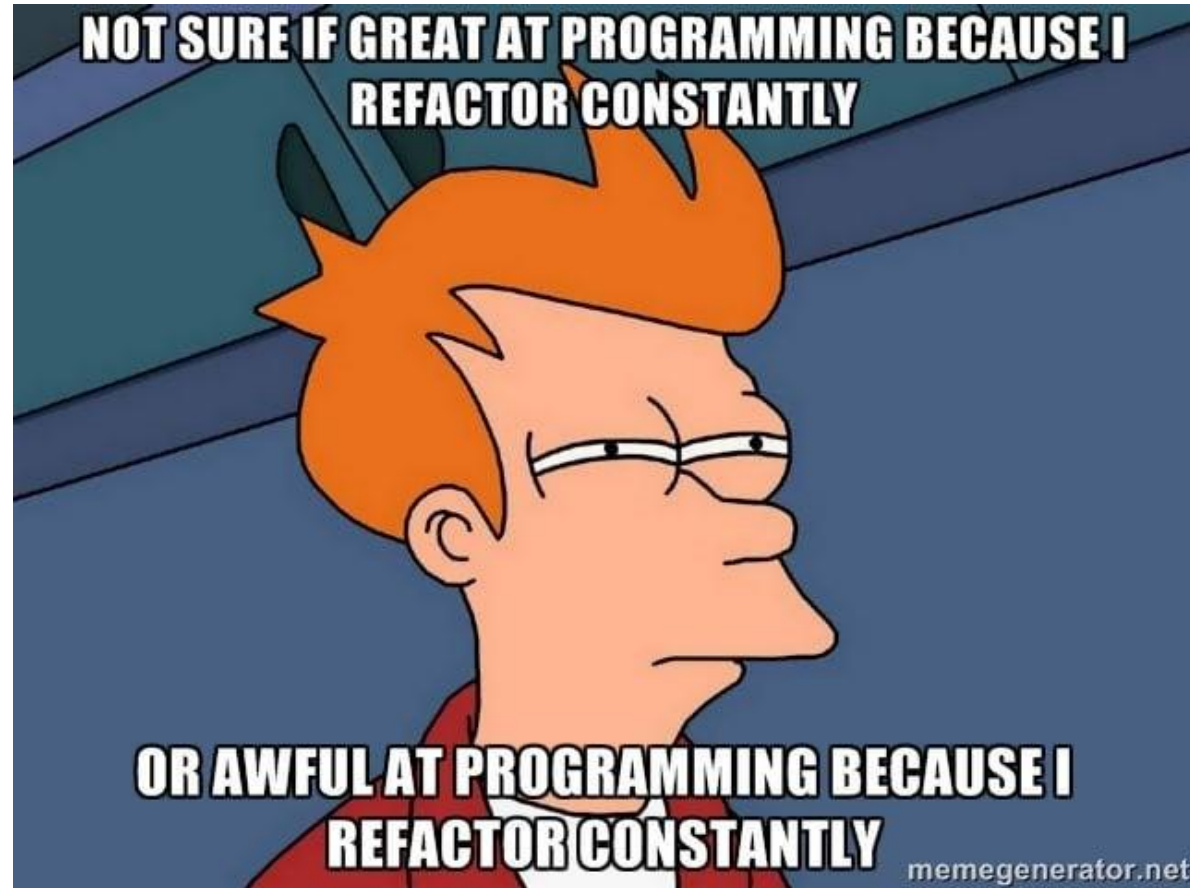
- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

Kapitel 2: Objektorientierung

1. Grundlagen
- 2. Header Files**
3. Vererbung
4. Polymorphismus

Exkurs: Git

Header Files: Einführung



Header Files: Einführung

Interface Trennung

- Es ist guter Stil das Interface (also nach den nach außen sichtbaren Teil) von der Implementierung zu trennen
- → .hpp und .cpp Dateien.
- .hpp enthält Interface, d.h. Funktions-Signaturen (Deklaration)
- .cpp enthält alle Implementierungen (Definition)

Header Files: Best practices

- Jedes .hpp file enthält einen *include guard* (mit beachteter *naming convention*)

```
Objektorientierung > 2.2 > hppcpp_concept > h++ R
1  #ifndef RECTANGLE_HPP_INCLUDED
2  #define RECTANGLE_HPP_INCLUDED
3
4  // My code
5
6  #endif // RECTANGLE_HPP_INCLUDED
```

- Alternative: `#pragma once` (Preprocessor Direktive, viel verbreitet, kein cpp Standard)
- Jedes .hpp und .cpp file inkludiert alle benötigten Header selbst und nutzt keine über andere header inkludierten Funktionen und Objekte
- .hpp files dürfen sich **nicht gegenseitig inkludieren** (*cyclic includes*)
- `#include <...>` // für externe Header
- `#include „...“` // für selbst geschriebene Header

Aufgabe: Header Files

Verwenden Sie die Aufgabenbeschreibung aus dem Git Repo:

2_Objektorientierung/215_header/VehicleV1_header/TASK_VEHICLE_HEADER.md

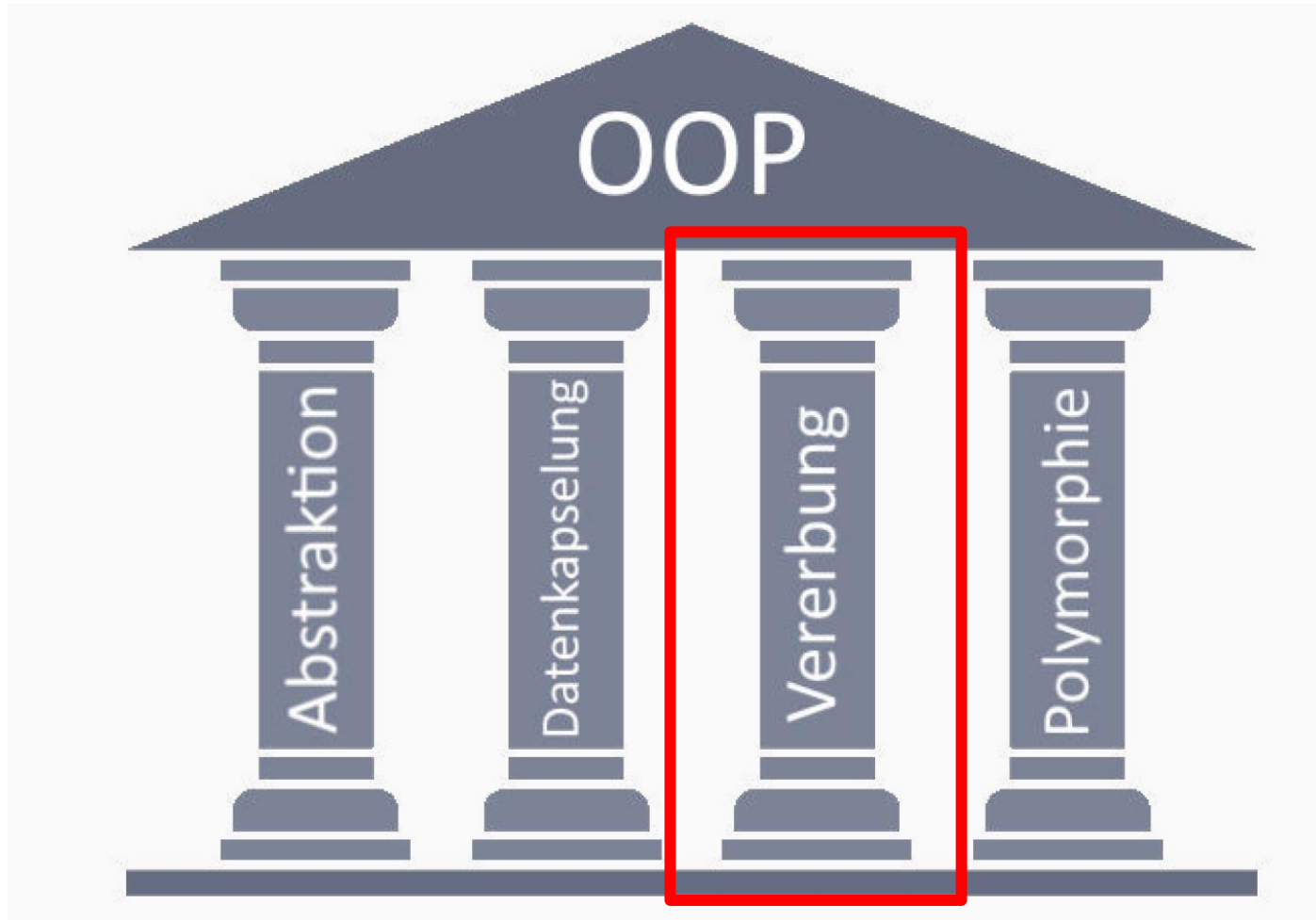
Testen Sie Ihre Klasse durch Aufruf in der `main` Funktion.

Kapitel 2: Objektorientierung

1. Grundlagen
2. Header Files
- 3. Vererbung**
4. Polymorphismus

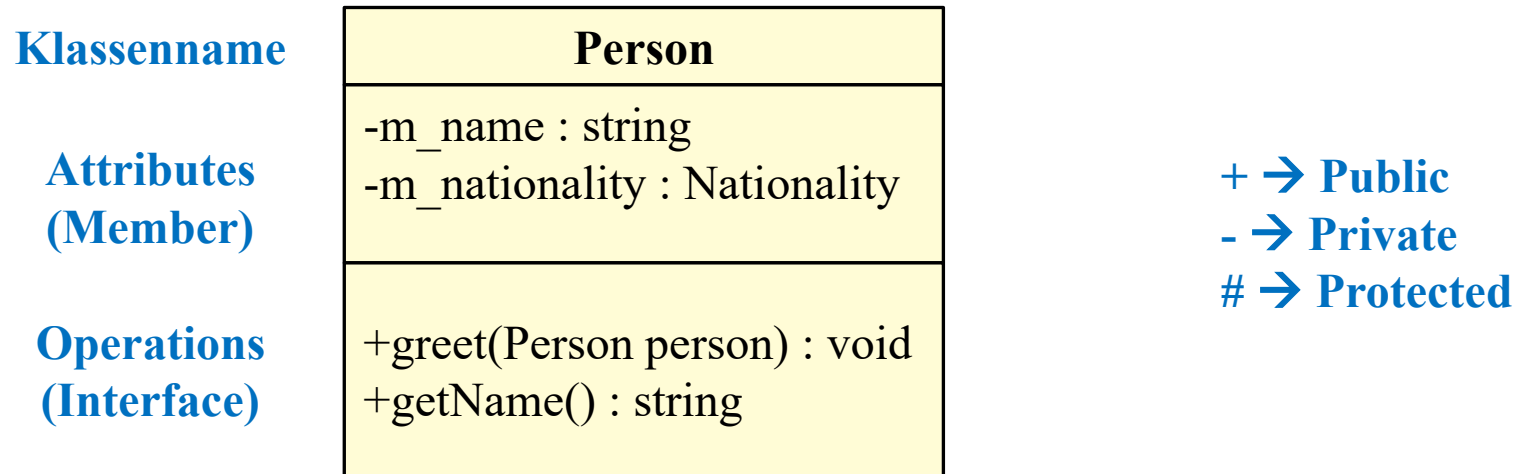
Exkurs: Git

Die vier Säulen der objektorientierten Programmierung



Einschub: UML – Unified Modelling Language

- Zur Visualisierung von Klassen und deren Zusammenhängen: UML-Klassendiagramme
- Bsp.: Klasse Person



UML - Motivation

- Modellierung von **has-a-Beziehung: Membervariablen!**
 - Bsp.: Eine Person hat einen Namen
- Modellierung von **is-a-Beziehung: Vererbung!**
 - z.B. ein Golf ist ein Auto, ein Student ist eine Person, ...
- Überblick UML Klassendiagramme [UML Class Diagram Tutorial](#)
- Vielzahl an Tools zur Modellierung verfügbar
 - Beispiel: [draw.io \(diagrams.net\)](#), [www.visual-paradigm.com](#), <https://www.plantuml.com/plantuml/uml/>
 - Manche können funktionalen Code erzeugen

Bsp: Personenverwaltung für die DHBW

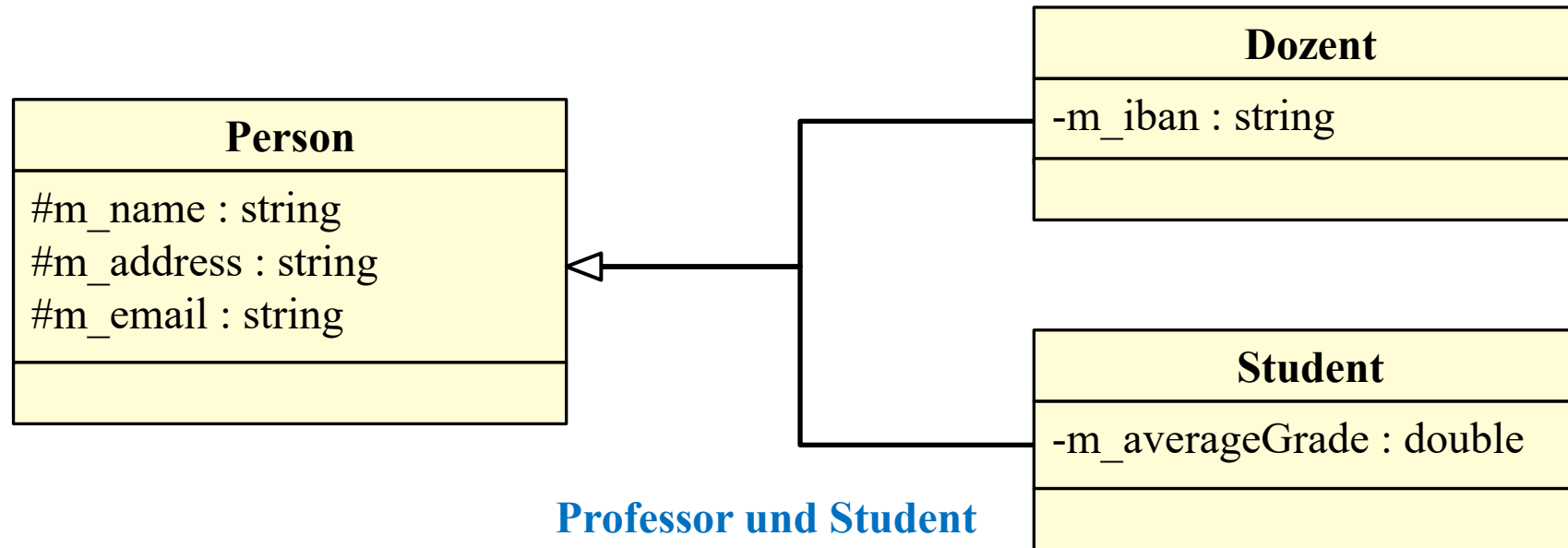
- Zwei Arten von Personen: Studenten und Dozent

Student
-m_name : string -m_address : string -m_email : string -m_averageGrade : double

Dozent
-m_name : string -m_address : string -m_email : string -m_iban : string

Bsp: Personenverwaltung für die DHBW

- Basis-Klasse: **Person** ← Kind-Klassen: **Professor**, **Student**



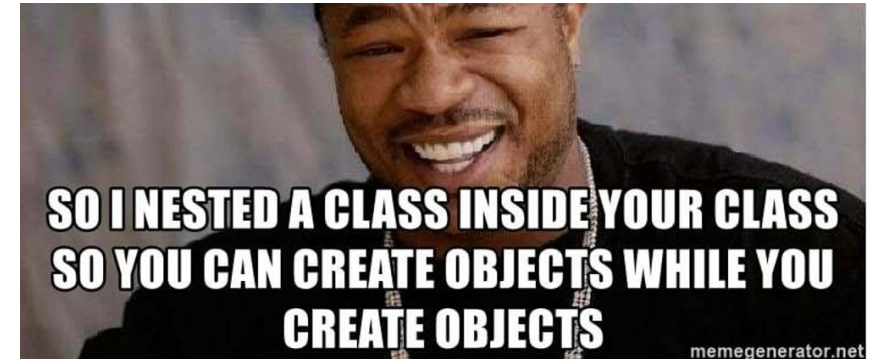
Professor und Student
leiten von Person ab
(is-a-Beziehung)

Möglichkeiten: Vererbung

- Verhindern von Code-Duplizierung: Überschneidende Funktionalität in eigene Klasse
- **Aber:** nur bei **echter is-a-Beziehung** Vererbung nutzen!
- Komplexität reduzieren:
 - Übersichtlichkeit durch kleinere Klassen
 - Neue Abstraktionsebene durch Verallgemeinerung (und damit kleinere Klassen)
- Ermöglicht erstellen eines gemeinsamen **Interfaces** (von außen sichtbarer Teil) von Klassen (kommt nochmal ausführlicher)

Nicht ohne Grund anwenden

- Kann die **Übersichtlichkeit** bei verschachtelter Vererbung verschlechtern
- Kann die **Wartbarkeit** verschlechtern → Elternklasse hat Einfluss auf Kinderklassen
- Am besten nicht ohne guten Grund anwenden: **KISS**: „*Keep it stupid simple*“

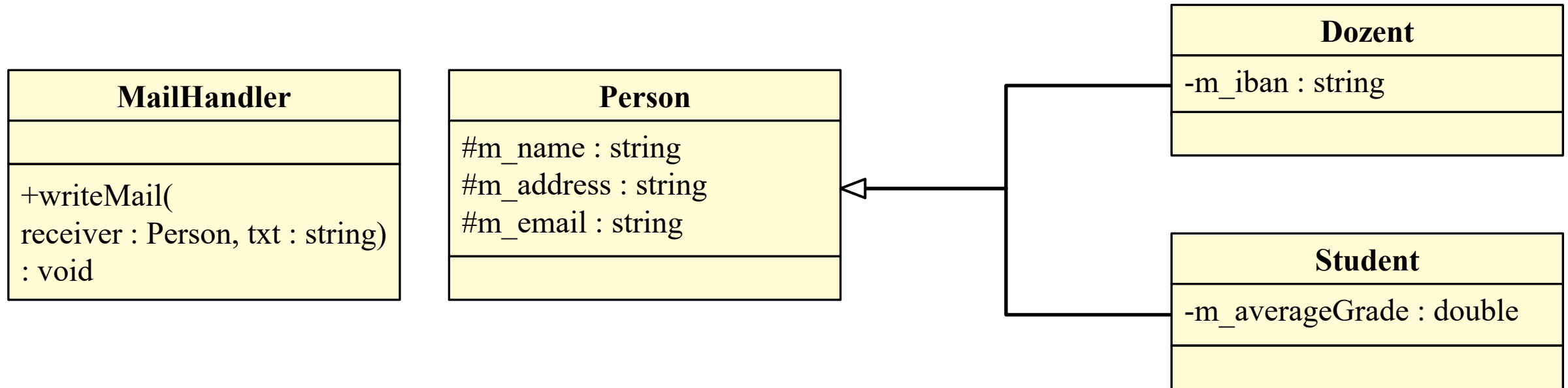


Was sind korrekte Ableitungen?

Fragen Sie sich „Jede/r <Kind-Klasse> ist ein <Basis-Klasse>.“

- Auto ← Sportwagen | Van | Combi
- Nahrungsmittel ← Süßigkeiten | Fleisch | Obst | Gemüse | Teigwaren | Getränke
- Viereck ← Parallelogramm ← Rechteck ← Quadrat
- Viereck ← Parallelogramm | Rechteck | Quadrat
- Quadrat ← Rechteck
- Bier ← Pils | Weizen | Export | Radler

Vererbung zur Generierung von Interfaces



Vererbung

Zugriffsmodifikationen: public/protected/private

Zugriffsmodifikationen

Access	public	protected	private
Same class	yes	yes	yes
Derived classes	yes	yes	no
Outside classes	yes	no	no

- Konstruktoren und Destruktoren werden bei Vererbung **nicht** übertragen

Art der Vererbung

- Public Inheritance (base class / child class):
 - `public` → `public`
 - `protected` → `protected`
 - `private` → **Nie** in der abgeleiteten Klasse verfügbar
- Protected Inheritance (fast nie genutzt)
 - `public, protected` → `protected`
- Private Inheritance (fast nie genutzt)
 - `public, protected` → `private`

Zugriffsmodifikationen dienen der Kapselung

Beispiele: Zugriffsmodifikationen

Frage: Was funktioniert?

```
1
2  class Base
3  {
4  public:
5      void a(){};
6  protected:
7      void b(){};
8  private:
9      void c(){};
10 };
11
12
13 int main()
14 {
15     Base base;
16     base.a();
17     base.b();
18     base.c();
19     return 0;
20 }
```

Beispiele: Zugriffsmodifikationen

Frage: Was funktioniert?

```
1  class Base
2  {
3  public:
4      void a(){};
5  protected:
6      void b(){};
7  private:
8      void c(){};
9  };
10
11 class Derived : public Base
12 {
13 };
14
15
16 int main()
17 {
18     Derived derived;
19     derived.a();
20     derived.b();
21     derived.c();
22     return 0;
23 }
```


Beispiele: Zugriffsmodifikationen

Frage: Was funktioniert?

```
1  class Base
2  {
3  public:
4      void a(){};
5  protected:
6      void b(){};
7  private:
8      void c(){};
9  };
10
11 class Derived : public Base
12 {
13     void test()
14     {
15         a();
16         b();
17         c();
18     }
19 };
20
21
```

Aufgaben: Vererbung Professor Student

Verwenden Sie die Aufgabenbeschreibung aus dem Git Repo:
**2_Objektorientierung/222_task_person_professor_student/
TASK_INHERITANCE_PROFESSOR_STUDENT.md**